That will lead you to design tests in a way that maximizes their usefulness and efficiency. You shouldn't aim to reach maximum test coverage but rather to write meaningful tests. For example, writing tests for standard getters and setters doesn't make sense.

An essential practice of continuous delivery is *test-driven development* (TDD), which helps achieve the goal of delivering software quickly, reliably, and safely. The idea is to drive software development by writing tests before implementing the production code. I recommend adopting TDD in real-world scenarios. However, it's not very suitable when teaching new technologies and frameworks in a book, so I won't follow its principles here.

Automated tests assert that new features work as intended and that you haven't broken any existing functionality. This means that automated tests work as *regression tests*. You should write tests to protect your colleagues and yourself from making mistakes. What to test and how in-depth to test is driven by the risk associated with a specific piece of code. Writing tests is also a learning experience and will improve your skills, especially if you're beginning your software development journey.

One way of classifying software tests is defined by the Agile Testing Quadrants model originally introduced by Brian Marick, and later described and expanded on by Lisa Crispin and Janet Gregory in their books *Agile Testing* (Addison-Wesley Professional, 2008), *More Agile Testing* (Addison-Wesley Professional, 2014), and *Agile Testing Condensed* (Library and Archives Canada, 2019). Their model was also embraced by Jez Humble and Dave Farley in *Continuous Delivery* (Addison-Wesley Professional, 2010). The quadrants classify software tests based on whether they are technology or business-facing and whether they support development teams or are used to critique the product. Figure 3.7 shows some examples of testing types I'll mention throughout the book, based on the model presented in *Agile Testing Condensed*.